



Computer System Engineering (CSL301)

LAB-1

2025-26 M

Introduction to Linux Tools

What we'll cover in lab sessions

01 Introduction to Linux Tools

02 Shell

03 Inter-process communication

04 Dynamic memory management

05 Thread Synchronization

06 File system

Change file permissions with chmod

- **chmod +x hello**

Adds execute permission for everyone – needed if ./hello fails with 'Permission denied'.

- **chmod u+x script.sh**

Adds execute permission for the file's owner (u) only.

- **chmod go-w secret.txt**

Removes write permission for the group (g) and others (o), leaving the owner's access unchanged.

- **chmod 755 hello.c**

Numeric form: owner gets read+write+execute (7), group and others get read+execute (5).

Numeric mode: r=4, w=2, x=1 per digit – add them up. 755 = rwxr-xr-x.

Watch what's running

- **top**

Displays running processes, CPU usage, memory usage, and system load in real time.

- **pstree**

Displays all running processes arranged in a tree structure by parent-child relationship.

- **ps**

Shows the processes running in the current terminal session.

- **man ps**

Opens the manual page for the ps command, documenting all of its options.

top is interactive and live-updating; press q to quit back to the shell.

Filter output with grep and pipes

- **`ps -ef | grep firefox`**

Pipes the full process list into grep, showing only lines that mention 'firefox'.

- **`ps aux | grep firefox | grep -v grep`**

Chains two greps — the second removes the grep command itself from the results.

- **`history | grep gcc`**

Searches your command history for every past use of gcc.

- **`cat /proc/cpuinfo | grep "model name"`**

Pipes a file's contents into grep to pull out just the matching lines.

/ sends one command's output straight into the next command's input — `grep -i` ignores case, `grep -c` counts matches.

Inspect the kernel through /proc

/proc is a virtual filesystem – it doesn't store files on disk, but exposes live kernel and process data as if it were a normal directory tree.

/proc/<pid>/status

Process Status

State, memory usage, and parent PID for a given process

/proc/<pid>/maps

Process Memory Map

The memory layout of a process's address space

/proc/interrupts

Interrupt Information

Hardware interrupt statistics per CPU

/proc/meminfo

Memory Information

Total, free, cached, and available system memory

/proc/cpuinfo

CPU Information

Processor model, cores, cache, and clock frequency

/proc/stat

System Statistics

CPU stats, interrupts, context switches, and boot time

Formatting and filtering process output

- **ps -ef**

Lists every running process in full format, across all users and terminals.

- **pstree -p**

Displays all processes as a tree, annotated with each process's PID.

- **ps -eo pid,ppid,stat,cmd**

Shows Process ID, Parent PID, process State, and the Command for every process.

- **ps -p <PID> -o pid,ppid,cmd**

Displays the PID, parent PID, and command name for one specific process.

- **pstree -p 1**

Shows the ASCII process tree rooted at PID 1 (usually systemd), with PIDs shown.

Combine -o with any field list to build a custom ps report tailored to what you need.

Learn about the machine you're on

- **lscpu**

Displays detailed CPU architecture info: number of CPUs, cores, threads, cache sizes, and model.

- **uname -a**

Displays kernel name, hostname, kernel version, architecture, and operating system.

- **uptime**

Shows how long the system has been running, logged-in users, and 1/5/15-minute load averages.

Shell builtins vs. executables

- **type cd**

Reports how the shell would run a command – e.g. 'cd is a shell builtin'.

- **which ls**

Searches your PATH for an executable named ls and prints its full pathname.

- **which -a ps**

Lists every matching executable found on PATH, not just the first one.

- **history**

A pure shell builtin – there is no /usr/bin/history executable to find.

Builtins (cd, history) run inside the shell itself; commands like ls or ps are separate executables the shell launches.

Reading VSZ and RSS in ps

Every process has a virtual memory size (what it thinks it has) and a resident set size (what's actually in RAM right now).

- `ps -o pid,vsz,rss,cmd -p <PID>`

Shows the PID, virtual size (VSZ), resident size (RSS), and command for one process.

VSZ

Virtual Size

Total address space the process has mapped, in KB – includes memory it has reserved but never touched.

RSS

Resident Set Size

The portion of that address space actually backed by physical RAM right now, in KB.

Measuring disk activity

- **iostat -x 1**

Prints extended disk statistics every 1 second – throughput, %util, and queue depth per device.

- **free -h**

Shows how much memory is currently used as buffer/cache before and after clearing it.

- **sudo sync; echo 3 > /proc/sys/vm/drop_caches**

Flushes dirty pages, then clears the page, dentry, and inode caches so the next read hits disk.

Reads served from the disk buffer cache never touch the disk – clear the cache first if you want to measure real I/O.

Tracing system calls with strace

- **strace ./hello**

Runs hello and prints every system call it makes to the kernel, with arguments and return values.

- **strace -c ./hello**

Suppresses the per-call trace and instead prints a summary: call counts, time spent, and errors per syscall.

- **strace -e trace=open,read ls**

Filters the trace to only the listed system calls – useful when the full output is too noisy.

A user-space program can't touch hardware or other processes directly – every one of those actions crosses into the kernel as a system call. strace makes that boundary visible.



```
$ echo "ready for the lab"
```

Next: the lab assignment

Apply these tools to real processes – CPU-bound loops, ancestor chains, memory usage, and disk I/O.